



Master 1 Informatique

# AI Models and Applications : Project Report

Celebrity Face Recognition Project

**BEL HADJ Asmaa**  
**VINCENT Valentin**  
**PHAM Kyllian**

**Superviseur :**  
**Enrico FORMENTI**

Université Côte d'Azur

Année 2025-2026

# Table des matières

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>2</b>  |
| 1.1      | Purpose of the Document . . . . .                      | 2         |
| 1.2      | Project Overview . . . . .                             | 2         |
| 1.3      | Outline of the Report . . . . .                        | 2         |
| <b>2</b> | <b>Extracting Bounding Boxes</b>                       | <b>3</b>  |
| <b>3</b> | <b>Faces Detection and Extraction using RetinaFace</b> | <b>4</b>  |
| 3.1      | Overview of RetinaFace Architecture . . . . .          | 4         |
| 3.2      | Dataset Preparation for Face Detection . . . . .       | 4         |
| 3.3      | Image Preprocessing and Cleaning . . . . .             | 5         |
| 3.4      | Accuracy Evaluation and Error Analysis . . . . .       | 5         |
| 3.5      | Output Generation and Data structuring . . . . .       | 5         |
| <b>4</b> | <b>Celebrity Recognition</b>                           | <b>7</b>  |
| 4.1      | Image Processing and Prediction . . . . .              | 7         |
| 4.2      | Models : VGG16, RESNET50, SENET50 . . . . .            | 8         |
| 4.3      | Test case and conclusion . . . . .                     | 8         |
| <b>5</b> | <b>GUI</b>                                             | <b>9</b>  |
| <b>6</b> | <b>Project Organization</b>                            | <b>11</b> |
| 6.1      | Workflow Management . . . . .                          | 11        |
| 6.2      | Task Distribution . . . . .                            | 11        |
| 6.3      | Code Management . . . . .                              | 11        |
| <b>7</b> | <b>Conclusion</b>                                      | <b>12</b> |
| 7.1      | Project Summary . . . . .                              | 12        |
| 7.2      | Problems encountered . . . . .                         | 12        |
| 7.3      | Areas for Improvement . . . . .                        | 12        |

# 1 Introduction

## 1.1 Purpose of the Document

This document aims to provide a detailed overview of the AI Models and Applications project. It outlines the challenges encountered, the solutions implemented, and the technical and the methodological choices that guided the development process.

## 1.2 Project Overview

This project, carried out as part of the AI Models and Applications course, focuses on recognizing celebrities in pictures based on their facial features. The project was coded in Python and by a group of three students : BEL HADJ BEL HADJ Asmaa, VINCENT Valentin and PHAM Kyllian.

The workflow is divided into three main steps. First, we use the yolol1n pre-trained network by Ultralytics to extract bounding boxes of people and store them in a dedicated “working” directory. Then, we employ the RetinaFace package to achieve pixel perfect bounding face extraction from these images. Finally, a pre-trained VGG network is used to identify the celebrities detected.

## 1.3 Outline of the Report

The structure of this report is organized as follows :

- The first section provides an introduction, the project overview, and the purpose of this document.
- The second section introduces the first step of the project : extracting bounding boxes using yolo11n. It details the dataset, the chosen methodology and the associated technological choices.
- The third section focuses on the second step of the project : detecting and extracting faces using RetinaFace. It explains the processing of the extracted images and evaluates the accuracy of the detection.
- The fourth section describes the final stage of the project : implementing the pre-trained model network for celebrity recognition. It outlines the integration process and discusses the obtained results.
- The fifth section presents the Graphical User Interface (GUI) designed for the project.
- The sixth section presents the organization of the project.
- The final section, the conclusion, summarizes the key findings and opens the discussion to potential improvements.

## 2 Extracting Bounding Boxes

The extraction of bounding boxes was realized using the Ultralytics library and the YOLO11n detection model. This model was chosen for its inference speed and simplicity of use.

YOLO (an acronym for You Only Look Once) is a real-time object detection system powered by deep learning. Unlike traditional methods that scan an image multiple times, YOLO processes the entire image in a single forward pass through the neural network. This allows it to simultaneously predict bounding boxes and class probabilities, making it exceptionally fast and efficient for high-speed applications without sacrificing significant accuracy.

The running pipeline is the following and is pretty straightforward :

1. **Model initialization** : The process begins by loading the pre-trained YOLO11n model weights.
2. **Inference execution** : A detection task is performed on an inputted image that contains one or multiple people.
3. **Data parsing** : The model will return an array containing multiple elements and their information such as the element's class number, coordinates, and more.
4. **Class filtering** : We keep only the elements of class number 0, containing the information needed to segment each human detected in the image.
5. **Region extraction and storage** : We extract all images using the coordinates and sizes provided in the array and save them into a directory named `img/working`.



FIGURE 1 – Example of bounding box extraction using YOLO11n.

### 3 Faces Detection and Extraction using RetinaFace

This section describes the implementation of the face detection and the extraction stage, which is handled in the Python script `extract_faces.py`. The objective is to detect faces within the person crops generated by YOLO (situated in the `working` folder) and to extract clean, well-framed face images for the recognition stage.

The following libraries were used :

- **RetinaFace** : face detection and landmark extraction.
- **PIL (Pillow)** : image loading and RGB conversion.
- **Numpy** : array manipulation and pixel-level processing.
- **os** : file and directory management.
- **json** : used for the serialization of the extracted metadata (age, emotion, ...).
- **DeepFace** : analyses facial characteristics using a pre-trained model.

#### 3.1 Overview of RetinaFace Architecture

As a requirement of this project, the RetinaFace model was used to detect and extract faces from the bounding boxes produced in the previous step. This model based on deep learning is widely recognized for its robustness and high accuracy. It detects faces with a high precision, delivers the position of the face as well as the facial landmarks (eye, nose,...) and a confidence score indicating detection reliability. RetinaFace analyzes an image at a pixel level, allowing it to locate faces accurately even if they are in a challenging position.

This method is essential for our project because it significantly reduces implementation complexity while ensuring high-quality detections. In my code, the function `detect_faces()` from RetinaFace is used to obtain a dictionary containing all the details of the faces detected. This is an example of what it return for a single face detected :

```
{
  "face_1": {
    "score": 0.9991,
    "facial_area": [36, 9, 60, 39],
    "landmarks": {
      "right_eye": [40.9346, 21.6634],
      "left_eye": [52.6751, 20.3137],
      "nose": [46.7876, 26.6579],
      "mouth_right": [42.7819, 31.8926],
      "mouth_left": [53.8709, 30.5876]
    }
  }
}
```

FIGURE 2 – Example output from RetinaFace : face detection with confidence score, facial area, and landmarks.

#### 3.2 Dataset Preparation for Face Detection

The images used at this stage come from the previous step, where yolo11n was used to detect persons and crop their bodies. These cropped images are stored in the “working”

folder.

Two main functions were implemented :

- `extract_all_faces_in_folder()` : iterates through all person images and processes them one by one.
- `extract_faces()` : detects and extracts faces from a single image.

The first function loops through the dataset and calls the second function for each image.

### 3.3 Image Preprocessing and Cleaning

Before extracting the faces, the raw detection output must be validated and cleaned. This step include :

- **Type verification** : ensure that the output returned by RetinaFace is a dictionary.
- **Confidence filtering** : keep only the detections with a confidence score superior or equal to 0.8 to ensure high-quality face results.
- **Boundary checking** : use the functions `max()` and `min()` to reframe the faces situated in the image borders. This avoids errors of dimension occurring during the extraction stage.
- **Padding addition** : add a padding of 15 percent around the detected face to improve lisibility and facilitate the cropping.

After these preprocessing steps, the faces are ready for extraction.

### 3.4 Accuracy Evaluation and Error Analysis

Although RetinaFace is highly robust, certain conditions can lead to hinder the analysis performed by DeepFace. The critical scenarios identified that may lead to failures in facial attribute extraction are the following :

- **Low Resolution** : if the extracted face is too small, the model may struggle to identify the features required for estimating age or emotion.
- **Extreme Poses** : full profile faces often result in reduced accuracy or complete non-detection by DeepFace.
- **Motion Blur** : this degrades the quality of the extracted attributes.
- **Overlapping** : when multiple faces overlap or are partially covered by objects, facial landmarks become difficult to localize.

To ensure the resilience of the processing step, a robust error-handling strategy was implemented. A `try...except` block was integrated directly into the extraction loop of the `extract_faces.py` script. This mechanism allows the program to catch exceptions raised by the method `DeepFace.analyse()` without interrupting the overall execution of the code. If the analysis fails for a specific face, the error is logged in the console, and the script automatically proceeds to the next image.

### 3.5 Output Generation and Data structuring

For extracting the faces, the Numpy array slicing is applied using the coordinates provided by RetinaFace. This approach isolates each detected face with precision. To add an extra feature to the project, the DeepFace library is used to extract additional characteristics such as the age, dominant emotion and dominant gender. This is an example of what it return for a single face detected :

```
[
  {
    "age": 30,
    "region": {
      "x": 4,
      "y": 10,
      "w": 33,
      "h": 34,
      "left_eye": null,
      "right_eye": null
    },
    "face_confidence": 0.95,
    "gender": {
      "Woman": 0.0429,
      "Man": 99.9571
    },
    "dominant_gender": "Man",
    "emotion": {
      "angry": 0.0,
      "disgust": 0.0,
      "fear": 0.0,
      "happy": 99.9998,
      "sad": 0.0,
      "surprise": 0.0,
      "neutral": 0.0002
    },
    "dominant_emotion": "happy"
  }
]
```

FIGURE 3 – Example output from DeepFace : facial attribute analysis.

The resulting data follow a rigorous hierarchical structure to ensure clarity and consistency.

- Each result is centralized in the **faces** directory located at the root of the project.
- Each processed image has a dedicated subfolder automatically created.
- Inside this subfolder, the data are the following :
  - an image containing the extracted face crop.
  - a JSON file containing the corresponding extracted characteristics.

This structure preserves a direct link between each face extracted and its metadata, thereby simplifying the next step of the project.

## 4 Celebrity Recognition

This section describes how the face recognition script has been implemented using **Keras**, **TensorFlow** and the **VGG Face** library. The goal is to identify a celebrity based on their face (which has already been cropped) using a pre-trained model from Oxford University. This model was originally developed in **Caffe** or **Torch**, both of which are challenging to implement in **Python**. The library **keras\_vggface** is the **Python** implementation of the Oxford model, but it has not been maintained for a long time. It took significant effort to correctly configure an environment where this library could run (**Python**, **NumPy**, **OpenCV-Python** and **TensorFlow** had to be set to specific versions). All requirements can be found in **requirements.txt**, but here are the most important libraries being used :

- **Keras** : High level API used to build and train neural network ;
- **TensorFlow** : Google’s open-source machine learning framework ;
- **keras\_applications** : A module that provides pre-trained deep learning models like RESNET, VGG16, SESNET ;
- **keras\_vggface** : library that provides pre-trained VGGFace model specifically designed for face recognition tasks ;
- **Numpy** : array manipulation and pixel-level processing ;
- **opencv-python** : tools for image and video processing, including reading/writing images.

To install the environment, **Anaconda** was used and an **INSTALL.md** file describes how to set-up the environment. The script that triggers the face recognition is called **face\_recognition.py**. This **Python** script implements a celebrity face recognition system using a pre-trained VGGFace deep learning model (other models can also be tested, such as RESNET50 and SENET50). The script processes a facial image and predicts which celebrity it most closely resembles from a database of known faces.

### 4.1 Image Processing and Prediction

The **predict\_celebrity()** function handles the entire prediction pipeline. It begins by loading an image from the specified file path using **OpenCV** and validates that the file exists. The image is then resized to  $224 \times 224$  pixels, which is the required input dimension for VGGFace models. The image data is converted to a floating-point array and expanded to include a batch dimension, transforming it from a 3D tensor to a 4D tensor (image dimensions, number of colors, and batch size) as expected by the neural network. Preprocessing is applied based on the selected model architecture. VGG16 uses version 1 preprocessing, while RESNET50 and SENET50 use version 2. This preprocessing step normalizes the pixel values according to the training procedure of each specific model, ensuring optimal performance. The preprocessed image is fed into the model to generate predictions, which are then decoded to retrieve the top celebrity matches.



## 4.2 Models : VGG16, RESNET50, SENET50

| Model           | Layers | Speed   | Parameters |
|-----------------|--------|---------|------------|
| VGG16           | 16     | Medium  | ~138M      |
| <u>RESNET50</u> | 50     | Fastest | ~25M       |
| <u>SENET50</u>  | 50     | Slowest | ~28M       |

FIGURE 4 – Comparison table of pre-trained models : VGG16, RESNET50, and SENET50.

A layer is step in face recognition analysis made by the neural network. It has only one job (for example analyses hair color, eye shape, etc). The more a model has layer the more accurate it should be. Parameters defines how score is calculated to match a celebrity face. For example : depending on hair darkness more point will be added to a celebrity score if the darkness matches. The number of parameters determines how the neural network memorize the faces it analyses. It can have a major impact since the number of parameters used determines the number of variations stored.

## 4.3 Test case and conclusion

In order to test our script, we used several pictures of celebrities. The three model were tested and only VGG face found the correct celebrity names. There is no cross references between both dataset (VGGFace used by VGG16 and VGGFace2 used by SENET50 and RESNET50). But in any case, only VGG16 did match the correct name for the pictures we tested. SENET50 and RESTNET50 failed constantly for pictures in their dataset. This may be explained by the number of parameters used by VGG16 (138 millions). Storing more variation of each celebrity appearance can explain why it matched better than the others even if it is not expected.

## 5 GUI

In order to have a way to implement the pipeline for people detection, facial segmentation and facial recognition, we decided to create a GUI to simplify the whole process for the user.

To have a simple yet complete GUI, we decided to use the streamlit library that is a simple library to handle.

The GUI is pretty simple and straightforward, the user just needs to choose the model and upload an image and in the backend, we will do all the actions :

1. Extract the bodies in the image
2. Extract the faces of every body detected
3. Predict the celebrity detected
4. Show the body, celebrity detected, the confidence of detection and some attributes (genre, age and emotion showed).

Here is what the GUI is looking like, When you start the script :

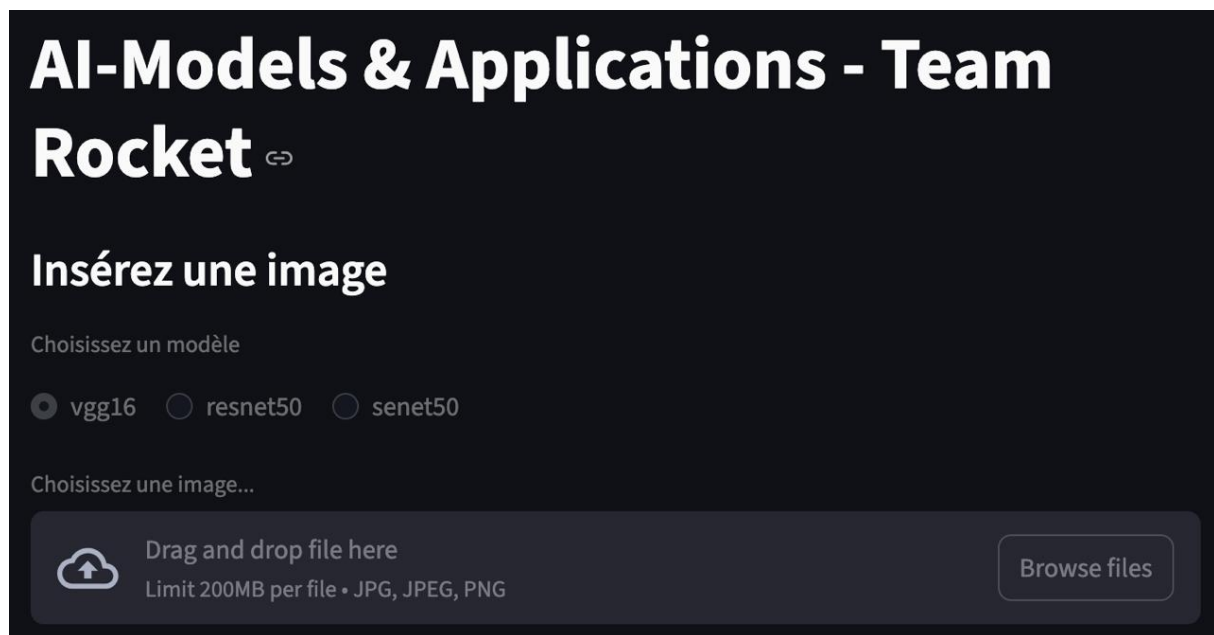


FIGURE 5 – Overview of the Graphical User Interface (GUI) developed with Streamlit.

When you give a picture :

# AI-Models & Applications - Team Rocket

## Insérez une image

Choisissez un modèle

☒ vgg16 ☐ resnet50 ☐ senet50

Choisissez une image...



Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



b.jpg 12.7KB



Fullscreen

## Résultats de l'analyse



## Résultats des prédictions

**Id:** 1

**Celebrity guessed:** Olivia Colman

**Confidence:** 4.17% (0.04173487052321434)

**Age:** 26

**Emotion:** happy

**Genre:** Man

FIGURE 6 – Display of analysis results and celebrity predictions within the GUI.

## 6 Project Organization

### 6.1 Workflow Management

To ensure smooth collaboration and rigorous code management, we relied on GitHub to organize our way of work. Our branching strategy was organized as follows :

- **Individual branches** : each team member worked on a dedicated branch to develop their features independently, preventing interference with the work of others.
- **“dev” branch** : this branch acted as an integration layer where we merged our respective contributions, tested modules and stabilized the application.
- **“main” branch** : reserved for the final, stable and deliverable version of the project.

### 6.2 Task Distribution

The project was divided into three parts for each team member :

| Team Member | Responsibilities                                                                                                       |
|-------------|------------------------------------------------------------------------------------------------------------------------|
| Valentin    | Body detection and segmentation using the YOLO model; development of the graphical user interface (GUI)                |
| Asmaa       | Precise face extraction and integration of facial feature analysis (extra feature analysis) using the DeepFace library |
| Kyllian     | Implementation and fine-tuning of pre-trained neural network for celebrity recognition                                 |

FIGURE 7 – Distribution of tasks and responsibilities among team members.

### 6.3 Code Management

Respecting the project’s requirements, we organize our code in the following way :

- **persons/** : This directory serves as the entry point, containing the untreated, raw images (.jpg) used for testing.
- **pre/** : This folder contains the preprocessing scripts, such as **bounding.py** for body extraction, **yolo11n.pt** (the YOLO model used by the bounding script to segment the bodies), and **extract\_faces.py** for facial extraction and attribute analysis.
- **rep/** : This directory is dedicated to the technical report and project documentation.
- **src/** : This folder contains the script for the Streamlit GUI **gui.py** and the celebrity recognition logic **face\_recognition.py**.
- **img/working** and **faces/** : These folders are dynamically generated during execution to store intermediate crops and final extracted face data with their respective JSON metadata.
- **faces\_resnet\_50/**, **faces\_senet\_50/**, and **faces\_vgg\_16/** : These folders contain images that work and don’t work for each model.

## 7 Conclusion

### 7.1 Project Summary

This project successfully designed and implemented a complete processing pipeline for celebrity recognition. By integrating YOLO11n for person detection, RetinaFace for high-precision face extraction and VGGFace for identity prediction, we transformed raw images into structured and analyzable data. Furthermore, the integration of attribute analysis and a graphical interface added a layer of depth to the project.

### 7.2 Problems encountered

The development of this project was marked by two major technical challenges :

- **Dependency Management** : managing dependencies proved particularly complex. Using older libraries alongside modern versions of `TensorFlow` and `NumPy` required the creation of isolated environments to ensure a stable system.
- **Input data quality** : recognition accuracy highly depended on the quality of the input images. Although `RetinaFace` is robust for detection, the recognition model remains sensitive to extreme poses and low resolution, which often lead to reduced confidence scores.
- **Poor result** : We had much more difficulty finding examples where the `RESNET50` and `SENET50` models give the correct result than with the `VGG16` model. The larger volume of data in the `VGGface2` dataset used by `RESNET50` and `SENET50` may be the cause.

### 7.3 Areas for Improvement

To further improve this project, several features could be considered :

- **Real-time Optimization** : The current pipeline processes static images. Adding support for video stream processing would be an interesting way to elevate the project.
- **Fine-tuning** : Retraining the final layers of `RESNET50` on a more contemporary celebrity dataset could improve its accuracy and potentially yield better results.
- **Advanced GUI** : Improving the current graphical interface by adding real-time comparative analytics between different models would provide a more informative and interactive user experience.